HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO GIỮA KỲ

Xây dựng Chatbot hỏi đáp về lịch sử Việt Nam sử dụng GraphRAG

Lớp: E22HTTT

Tên: Nguyễn Đức Anh

ID: B22DCDT017

Người hướng dẫn: Kim Ngọc Bách

Hà Nội – 2026

Báo Cáo Tuần 7

Building the Pipeline to Process Raw Data into a Knowledge Graph for the RAG System

1. Executive Summary

Currently, the project has completed 100% of the core Data Pipeline phase. The system is now capable of automating multi-threaded document reading (PDF, TXT), data cleaning, context-aware text chunking, and applying AI (Gemini Flash) to successfully extract Entities (Nodes) and Relationships (Edges).

2. Details of Completed Milestones

Phase 2.1: Data Collection and Cleaning

- Data Sources: The system is processing a variety of document sources, including textbooks, articles, Wikipedia, and the Law Library (Thư viện Pháp luật).
- PDF Processing: Successfully integrated the PyMuPDF (fitz) library to extract text from PDF files.
- Noise Filtering (Data Cleaning): Built filters using Regex and string replacement functions to completely remove watermarks, as well as personally identifiable information with font errors generated during the document download process.
- Result: Documents have been standardized into clean, plain text, creating a strong foundation for the AI model's reading comprehension.

Phase 2.2: Data Fragmentation - Chunking

- Technology Used: Applied RecursiveCharacterTextSplitter from the LangChain library.
- Optimized Configuration for RAG:
    - Chunk size (800 characters): Ensures each text snippet contains sufficient context of an event or battle without obscuring information.
    - Chunk overlap (150 characters): Retains the tail end of the previous chunk and appends it to the beginning of the next one. This prevents breaks in the historical flow and ensures the AI does not miss causal relationships between sequential events.

- Result: All .txt files in the directory have been scanned and split, with their structure and source information saved into the processed_chunks.json file. Retaining the source field is crucial so that the future Chatbot system can cite reference materials for users.

Phase 2.3: Knowledge Graph Extraction

- Model: Utilized Google GenAI (gemini-flash-latest) with the standard JSON output enforcement feature (response_mime_type="application/json").
- Prompt Engineering: Clearly defined extraction rules:
    - Nodes: Strictly limited to the following categories: Figures (People), Locations, Organizations, Events, and Weapons.
    - Edges: Must be directed and include action descriptions to form accurate Triplets (Subject - Predicate - Object).
- Result: The final graph data has been successfully synthesized into the extracted_graph_data.json file.

3. Next Steps:

1. Ingest Data into Database: Write a Python script using the Neo4j driver (e.g., the neo4j package) to read the extracted_graph_data.json file.
    - Use Cypher queries (such as MERGE) to push all these Nodes and Relationships to Neo4j AuraDB or a local database.
2. Integrate LangChain GraphCypherQA: Once the data is in Neo4j, the next step is to build the Chatbot Backend module, which will translate natural language user queries into Cypher queries to search for historical information on the graph.